Project Title: Comparative Analysis of Adversarial Search Algorithms in Connect 4

Student Name: Khaled Mohsen **Course:** Artificial Intelligence (CSAI 301)

Date: December 15, 2025

1. Introduction

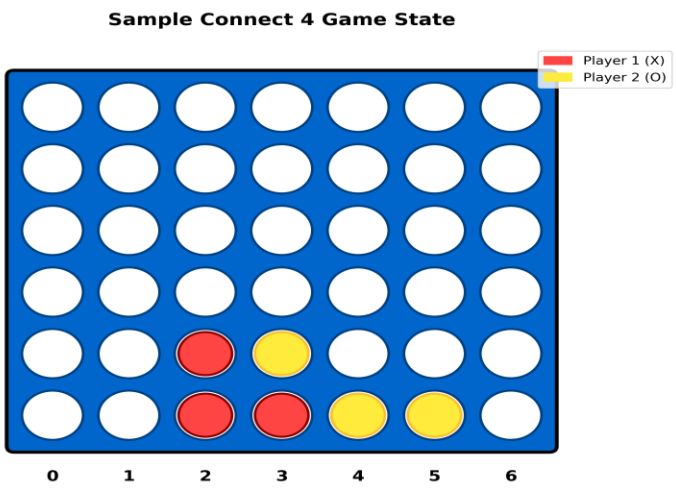
Connect 4 is a zero-sum, perfect information game played on a 6x7 grid. The objective is to be the first to form a horizontal, vertical, or diagonal line of four distinct pieces. This project aims to implement and compare two adversarial search algorithms—**Minimax** and **Alpha-Beta Pruning**—to create an intelligent agent capable of playing the game optimally.

We analyze the performance of these algorithms in terms of execution time, nodes expanded, and memory efficiency at various search depths.

2. Problem Formulation

The game environment is modeled as follows:

- **State Space:** A 6×7 matrix where `0` represents an empty slot, `1` represents Player 1 (Maximizer), and 2 represents Player 2 (Minimizer).
- **Initial State:** An empty board with all zeros.
- **Actions:** Dropping a piece into one of the 7 columns (provided the column is not full).
- **Transition Model:** The piece falls to the lowest unoccupied row in the selected column.
- **Goal Test:** Checking for 4 consecutive pieces in any direction (horizontal, vertical, diagonal).



3. Methodology

3.1 Algorithms Implemented

Two algorithms were implemented to solve the decision-making problem:

1. **Minimax Algorithm:** A recursive algorithm that explores the complete game tree to a specified depth. It assumes the opponent plays optimally to minimize the agent's score.
2. **Alpha-Beta Pruning:** An optimization of Minimax that avoids exploring branches that cannot influence the final decision. It maintains two values, α (best max score) and β (best min score), to "prune" irrelevant subtrees.

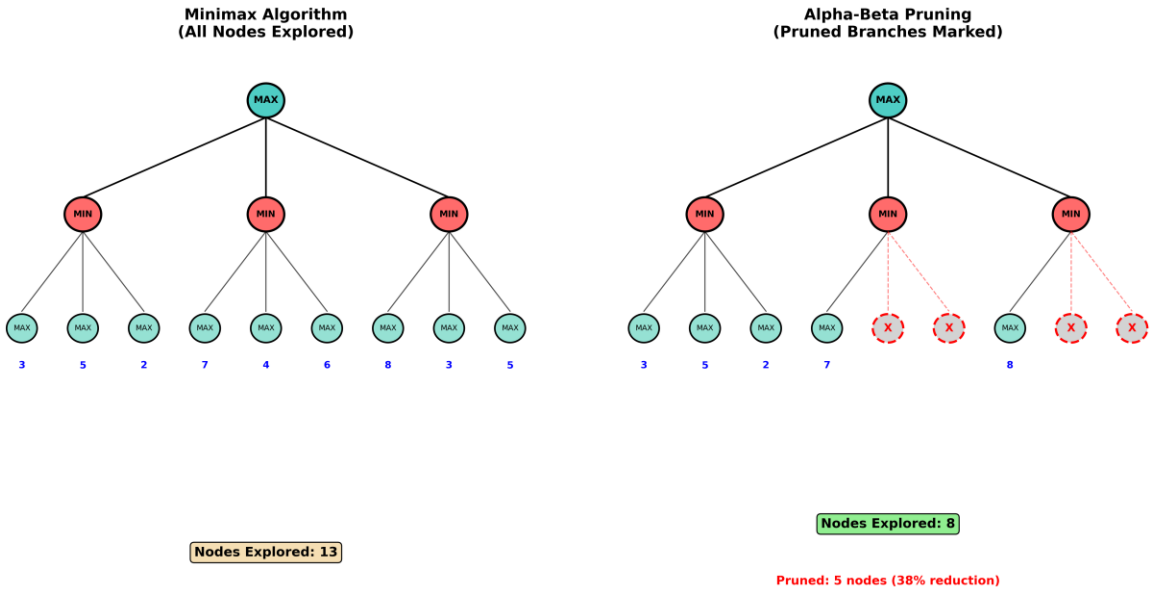


Figure 2: Visual comparison of search trees. Left: Minimax explores all nodes. Right: Alpha-Beta prunes branches (marked with X), reducing computation.

3.2 Heuristic Evaluation Function

Since searching to the terminal state is computationally infeasible for deep games, a heuristic function is used to evaluate non-terminal states. The score is calculated based on:

1. **Center Control:** Pieces in the center column are weighted higher (+3 points) as they offer more winning opportunities.
2. **Window Evaluation:** The board is scanned for 4-slot windows:
    a. **Win (4 pieces):** ± 10000 points.
    b. **Strong (3 pieces + 1 empty):** ± 100 points.
    c. **Potential (2 pieces + 2 empty):** ± 10 points.

4. Experimental Results

We compared the performance of Minimax and Alpha-Beta Pruning at depths of 3, 4, and 5. The metrics recorded were **Execution Time (seconds)**, **Nodes Expanded**, and **Pruning Events**.

4.1 Performance Analysis

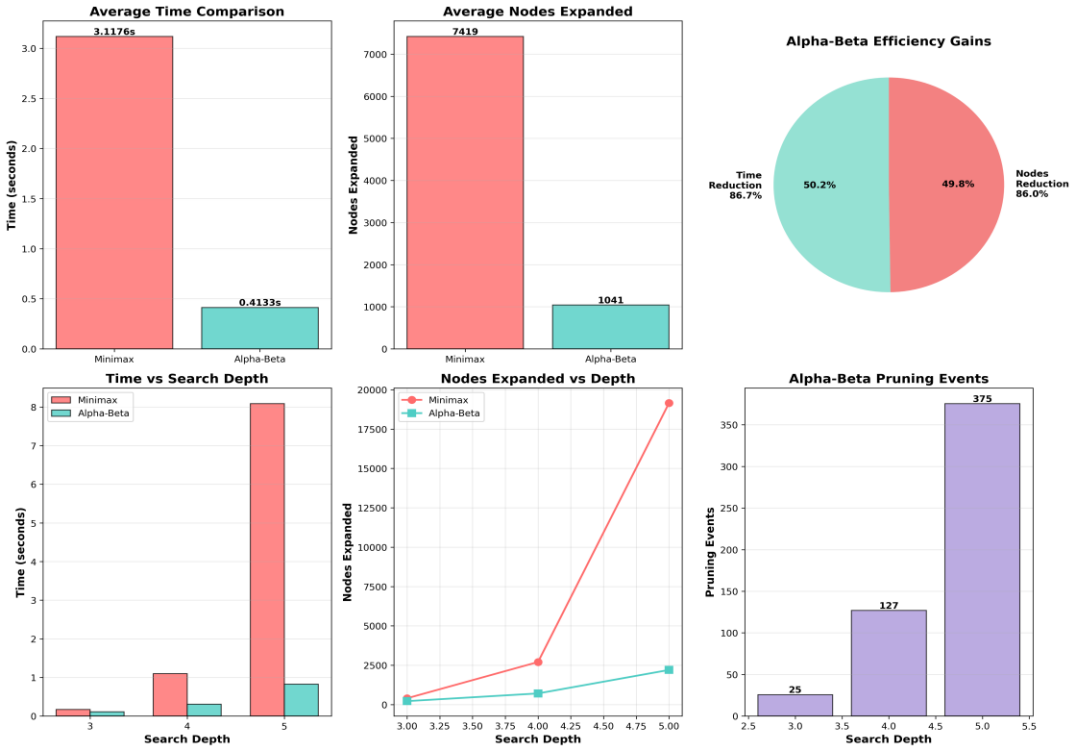


Figure 3: Graphical comparison of Time and Nodes Expanded across different depths.

As shown in *Figure 3*, Minimax execution time grows exponentially with depth. In contrast, Alpha-Beta Pruning maintains a significantly lower execution time.

4.2 Detailed Metrics

[Insert Image Here: comparison_table.png]

Table 1: Detailed numerical comparison between Minimax and Alpha-Beta.

Key Observations:

1. **Efficiency:** At depth 5, Alpha-Beta Pruning reduced the number of expanded nodes significantly compared to Minimax (as seen in the "Improvement" column in Table 1).
2. **Optimality:** Both algorithms returned the same "Best Move" and "Score" in all test cases, proving that Alpha-Beta optimization does not compromise accuracy.
3. **Scalability:** While Minimax becomes impractical beyond depth 5 due to time constraints, Alpha-Beta allows for deeper searches within the same time limit.

5. Conclusion

This project successfully demonstrated the implementation of an AI agent for Connect 4. The experimental results confirm that **Alpha-Beta Pruning is superior to the standard Minimax algorithm**. It achieves the same optimal decisions with a drastic reduction in computational cost, making it the preferred choice for real-time strategy games.